Digital Product Passport — Overview

- Full-stack DPP: back office → publish → QR → public passport + scans.
- Stack: FastAPI + Postgres + Alembic · Next.js + MUI + RTK Query.
- Full write-up: docs/REPORT.md (architecture, security, load, demo).

Run locally

- 1. Copy env examples; docker compose up db
- 2. backend: uv sync, alembic upgrade head, seed_*, uvicorn
- 3. frontend: npm i && npm run dev
- 4. App :3000 · API docs :8000/docs
- Logins: admin@example.com / admin1234 · editor@example.com / editor1234

Architecture choices

- Separate passports table with stable public_uuid (QR-safe).
- Republish bumps version, keeps UUID; soft delete + retention purge.
- Shared editor workspace; roles admin|editor; Storage protocol local/MinIO.
- Prod API: Gunicorn + Uvicorn workers; optional Redis cache.

Security highlights

- JWT access + httpOnly rotating refresh; reuse of old refresh revokes sessions.
- IP rate limits (auth / public / api) with 429 + Retry-After.
- Uploads: size cap + magic bytes; keys under products/{id}/ only.
- Access token in memory on the client (not localStorage).

Load / stress

- seed_load — bulk LOAD-* products for list/dashboard pressure.
- stress_test — concurrent GETs; writes docs/load-results.md.
- For pure throughput set RATE_LIMIT_ENABLED=false during the run.

Tests & CI

- Backend: pytest (dpp_test). Frontend: Vitest + production build.
- GitHub Actions on main/master: backend + frontend jobs.